

Transition: DS + Algorithms

Vectors +
Lists



First: Worksheet

Linked Lists vs. Arrays

Both store "a list".

Operations:

- begin
- end
- empty
- front
- back
- insert
- delete

So: what's
the difference?

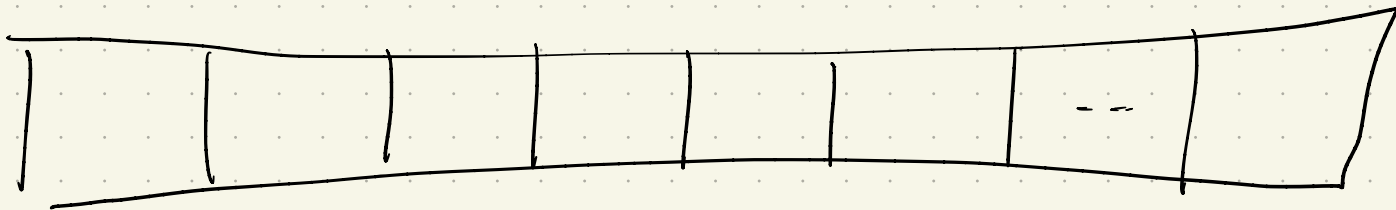
Arrays or Vectors:

Assume data is stored in a
contiguous region:

Need to know:

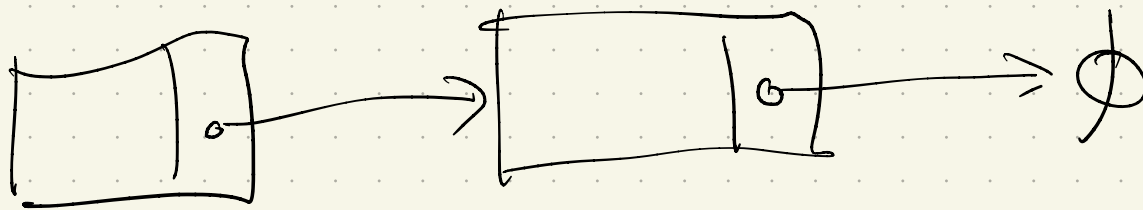
•

•



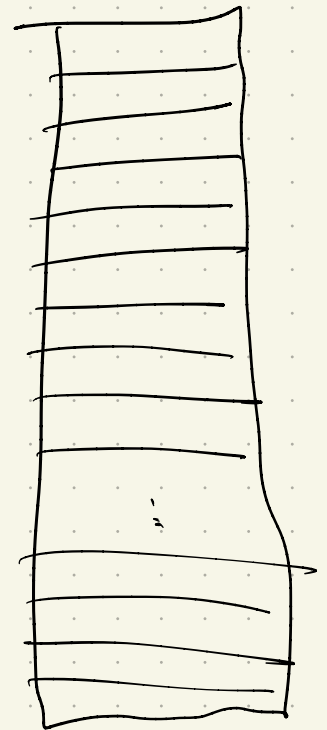
Linked lists:

Not contiguous $\rightarrow$ instead, each element stores address of the next one!



Need:

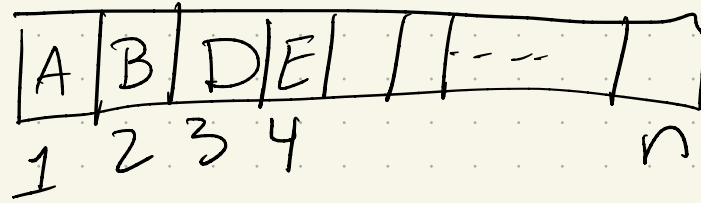
In memory



Trade-offs:

Consider $\text{insert}(C, 3)$:

Vector



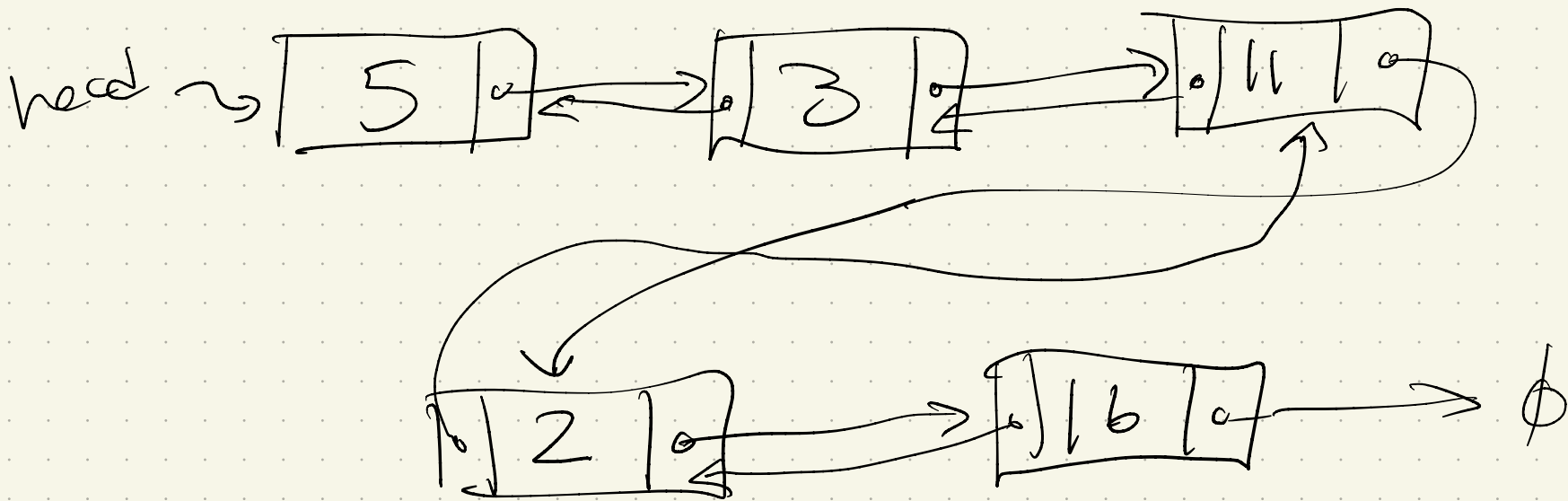
What if full?

Similar issue with delete:

5	2	11	3	6	-		
1	2	3	4	5			n

delete(3)

In lists:



delete 3:

In lists, recursion comes up
naturally:

delete(i):

(You saw this with length.)

Some special cases:

Inserting at the end of a vector:

Inserting or deleting in a list
given a pointer to the node:

Run times

	List	Vector
insert		
insert at end		
access position i		
delete		

Amortized analysis

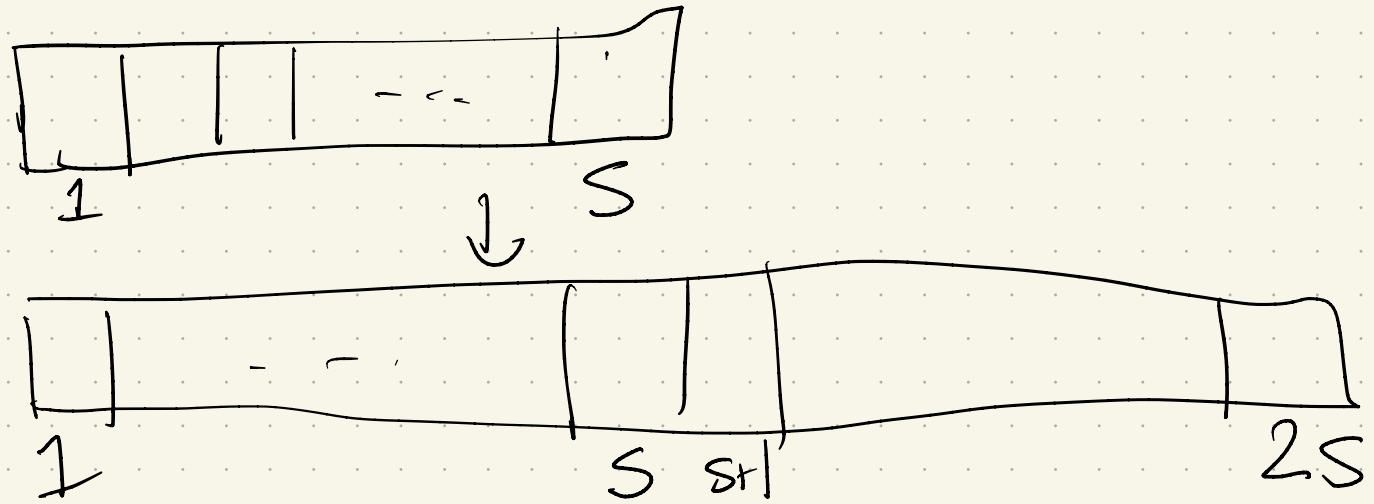
If we insert at end of a vector
 n times:

Except: When "full", can make larger
array & copy down, so we have
room.

Time:

So: what is time for n insertions
at end of a vector?

Note: Every time we double array
size, we have more room!



Formalize: what is average cost, if we
do many?

Claim: If we start with an empty vector, total time to do n insertions at end is $O(n)$.

proof:

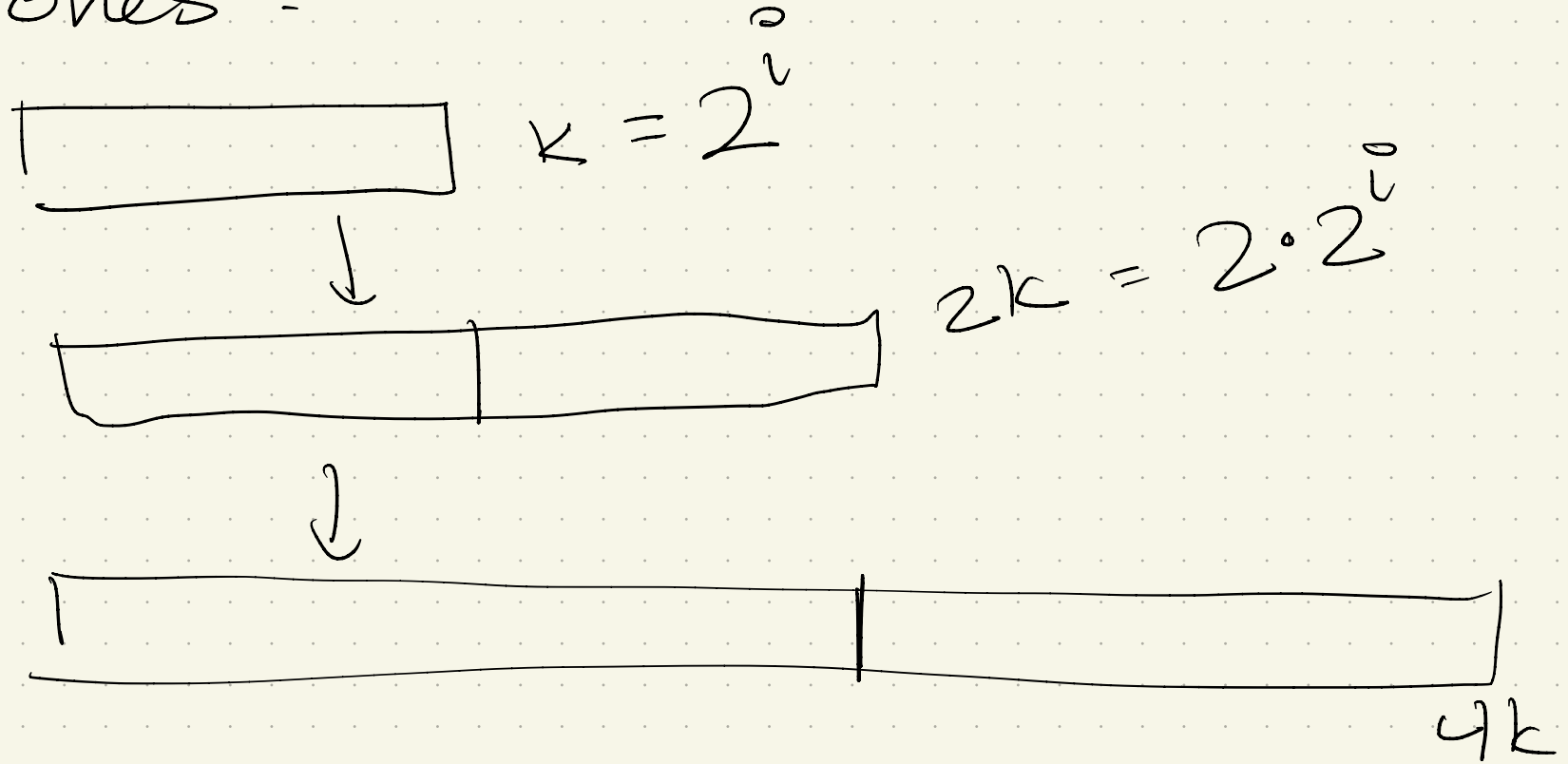
use "accounting" method:

Each $O(1)$ operation costs \$1.

So non-overflow insert:

& overflow insert:

Then, overcharge the non-overflow
ones:



Implications:

Which to use?